Introduction to Spring & SpringBoot

1. The Starting Point: How Web Applications Communicate

Before understanding Spring, Spring Boot, Servlet, or any backend framework, we first need to understand one basic question:

> How does a user sitting on one machine communicate with code running on another machine?

This is the foundation of web development.

When you open a browser and type:

```
www.amazon.com
```

your browser is running on your laptop or mobile phone, while Amazon's application is running on Amazon's server somewhere else.

At the most basic level:

```
Your Browser  ---- talks to ----  Amazon Server
```

This structure is called **Client–Server Architecture**.

---

2. Client–Server Architecture

What is a Client?

A **client** is the side that asks for something.

Examples of clients:

```
Browser
Mobile app
Postman
Frontend React app
Android app
iOS app
```

When you open a website, your browser becomes the client.

It sends a request like:

> "Hey server, give me this webpage."

What is a Server?

A **server** is the side that receives requests, processes them, and sends back a response.

Examples of servers:

```
Amazon server
YouTube server
Bank server
Your Spring Boot application
```

A server usually performs tasks like:

```
Checking login details
Fetching data from a database
Applying business rules
Saving information
Returning a response
```

The basic idea is simple:

```
Client asks.
Server responds.
```

---

3. HTTP: The Language of the Web

Now the important question is:

> How does the browser know how to ask?
>
> How does the server understand what the browser is asking?

For communication to happen properly, both sides need a common language.

That common language is called **HTTP**.

What is HTTP?

HTTP stands for:

```
HyperText Transfer Protocol
```

In simple words:

> HTTP is the rulebook for communication between a client and a server.

HTTP defines:

```
How a request should look
How a response should look
Which method is being used
Which URL is being called
What data is being sent
Which status code is returned
```

So the browser and server do not randomly exchange text. They follow a proper format.

HTTP is an **application-layer protocol** that works on top of TCP/IP.

---

4. Request–Response Cycle

Every web interaction follows the same basic pattern:

```
Client sends request
Server processes request
Server sends response
Client displays or uses the response
```

Example:

```
www.coderarmy.in/courses
```

The browser may send an HTTP request like:

```
GET /courses
Host: www.coderarmy.in
```

The server receives the request and understands:

```
The user wants the courses page.
Fetch the courses data.
Prepare the response.
Send it back to the browser.
```

The server may send back:

```
Status: 200 OK
Body: course data / HTML page / JSON
```

The browser receives the response and displays the page.

---

5. Anatomy of an HTTP Request

An HTTP request usually contains four main parts:

```
Method
URL or path
```

```
Headers
Body
```

Example:

```
POST /login
Content-Type: application/json

{
  "email": "abc@gmail.com",
  "password": "12345"
}
```

Meaning:

```
POST        → I am sending data
/login      → I want to call the login functionality
Headers     → Extra information about the request
Body        → Actual data being sent
```

HTTP Methods

The first line of an HTTP request is called the **request line**.

It contains the method, path, and HTTP version.

The method tells the server what action the client wants to perform.

Method	Meaning	Example Use
`GET`	Read data	Fetch a list of orders
`POST`	Create data	Place a new order
`PUT`	Replace data completely	Update an entire user profile
`PATCH`	Update data partially	Change only the phone number
`DELETE`	Remove data	Cancel an order

HTTP Headers

Headers are key-value pairs that provide extra information about the request.

They tell the server things like:

```
What format the client can understand
What format the request body is in
Who the client is
Which host the client is trying to reach
```

Common examples:

```
Accept: application/json
Content-Type: application/json
Authorization: Bearer token
Host: www.coderarmy.in
```

Headers are very important in real Spring applications because we frequently work with authentication, JSON data, API communication, and request metadata.

HTTP Body

The body carries the actual data being sent by the client.

It is commonly used with:

```
POST
PUT
PATCH
```

Example body:

```
{
  "name": "Rohit",
  "email": "rohit@example.com"
}
```

In modern APIs, the request body is usually sent in JSON format.

`GET` requests usually do not have a body because they are mainly used to fetch data.

---

6. Anatomy of an HTTP Response

An HTTP response usually contains:

```
Status code
Headers
Body
```

Example:

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "message": "Login successful"
}
```

The most important part of the response is the **status code**.

Status Code	Meaning
`200`	Request successful
`404`	Resource not found
`500`	Internal server error

We will study status codes in more detail later.

---

7. JVM: A Java Program Runs Locally

Now let's compare this web communication model with a normal Java program.

Example:

```java
public class Main {
    public static void main(String[] args) {
```

```
        System.out.println("Hello World");
    }
}
```

When we run this program:

```
java Main
```

it runs inside the **JVM**, which means **Java Virtual Machine**.

Your `.java` source file is compiled by `javac` into platform-independent bytecode stored in `.class` files.

The JVM loads this bytecode and executes it.

This is the reason Java is known for:

```
Write once, run anywhere
```

The same bytecode can run on Windows, macOS, or Linux as long as the JVM is available.

---

8. Normal Java Program vs Web Application

A normal Java program usually follows this pattern:

```
Start
Run instructions
Finish
Exit
```

But a website or backend application follows a very different pattern:

```
Start
Keep running
Wait for requests
Process requests
```

```
Send responses
Continue running
```

A website is not like a program that runs once and exits.

A web server is a program that stays alive continuously and keeps listening for incoming requests.

---

9. Why Core Java Alone Is Not Enough for Web Applications

Core Java understands concepts like:

```
Classes
Objects
Inheritance
Collections
Threads
Files
```

But Core Java does not automatically understand web concepts like:

```
HTTP requests
URLs
Headers
Cookies
Sessions
REST APIs
```

These are web-related concepts, not basic Java language concepts.

Even if we keep a Java program running using something like:

```java
while (true) {
    // keep program alive
}
```

it still does not automatically understand a request like:

```
GET /hello
```

Someone has to read that request, understand it, and map it to the correct Java logic.

---

10. Can Java Open Network Connections?

Yes, Java can do networking.

Java has had the `java.net` package since Java 1.0.

For example:

```java
ServerSocket server = new ServerSocket(8080);
```

This allows a Java program to listen on port `8080`.

So technically, Java can communicate over a network.

But there is still a problem.

When a browser or Postman sends an HTTP request, Java receives a raw TCP connection — basically a stream of bytes.

Example request:

```
GET /users HTTP/1.1
Host: localhost:8080
```

To the JVM, this is just data.

It does not automatically understand:

```
GET means fetch data
/users is an endpoint
Host is a header
```

Someone has to interpret all of this manually.

---

11. What We Would Need to Do Manually in Core Java

If we tried to build a web application using only Core Java, we would have to handle many things ourselves.

We would need to:

1. Open a port using `ServerSocket`
2. Read raw input streams
3. Parse the HTTP request manually
4. Extract the method, URL, headers, and body
5. Route the request to the correct Java logic
6. Create the HTTP response in the correct format
7. Manage multiple users using threads
8. Handle errors, malformed requests, and connection behavior

Example of manual routing:

```
if (url.equals("/users")) {
    // call user logic
} else if (url.equals("/orders")) {
    // call order logic
}
```

This code has nothing to do with actual business logic.

It is repeated technical work that every Java web application would need.

That is why a standard solution was needed.

---

12. The Problem Before Servlets

We had this gap:

```
Browser
   |
HTTP Request
   |
   v
 ???
   |
Java Code
```

The browser sends HTTP requests.

Java contains classes and methods.

But something is needed between HTTP and Java to translate web requests into Java method calls.

For example, if a browser sends:

```
GET /hello HTTP/1.1
Host: localhost:8080
```

how should this request trigger a Java method like:

```
sayHello();
```

This translation is handled by:

```
Servlet Container
Servlet
```

---

13. Java Servlets

A **Servlet** is a Java object that can handle HTTP requests.

In simple words:

> A Servlet is a special Java class designed for web applications.

Conceptually:

```
HTTP Request
      |
      v
   Servlet
      |
      v
  Java Code
```

Servlets were one of the first standard Java technologies created specifically for building web applications.

A Servlet runs inside a **Servlet Container**.

Examples of Servlet Containers:

```
Apache Tomcat
Jetty
Undertow
```

---

14. What Does a Servlet Container Do?

A Servlet Container sits between the outside web world and your Java code.

It handles the low-level web work for you.

A Servlet Container is responsible for:

```
Opening a port such as 8080
Listening for HTTP requests
Reading TCP bytes
Parsing HTTP requests
Creating request and response objects
Managing threads
Calling servlet methods
Sending HTTP responses
Handling connection behavior
```

Instead of manually reading bytes and parsing HTTP text, the Servlet Container gives us Java-friendly objects.

For example, it can call methods like:

```
doGet()
doPost()
```

So the request:

```
GET /hello
```

can eventually be handled by Java code inside a servlet.

---

15. Why Was Spring Needed Then?

Servlets solved an important problem.

They made it possible for Java applications to handle HTTP requests properly.

But building large enterprise applications directly with Servlets became difficult over time.

Common problems included:

```
Too much boilerplate code
Too many configurations
Tight coupling between classes
Difficult testing
Difficult maintenance in large applications
Repeated code across projects
```

As applications became bigger, developers needed a better way to organize code, manage objects, handle dependencies, and reduce configuration complexity.

This is where the **Spring Framework** became important.

---

16. What is Spring Framework?

Spring Framework is one of the most popular frameworks in the Java world.

It was created to make enterprise Java development easier, cleaner, and more maintainable.

Spring introduced important concepts like:

```
IoC
Dependency Injection
Bean Management
Configuration
Loose Coupling
```

We will study these concepts in depth later in the series.

For now, remember:

> Spring helps us build Java applications in a cleaner and more manageable way.

---

17. Spring is an Ecosystem

Spring is not just one small library.

Spring is a large ecosystem of projects and frameworks.

Different Spring projects solve different problems.

Some important parts of the Spring ecosystem are:

```
Spring Core
Spring MVC
Spring Data
Spring Security
Spring AOP
Spring Boot
Spring AI
```

---

18. Spring Core

Spring Core is the foundation of the Spring ecosystem.

It provides the most basic and important features of Spring.

Spring Core includes:

```
IoC
Dependency Injection
Bean Management
Configuration
ApplicationContext
```

Without Spring Core, other Spring projects would not exist.

Spring Core is the base on which many other Spring modules are built.

---

19. Spring MVC

Spring MVC is used to build web applications and REST APIs.

It is built on top of:

```
Servlets
+
Spring Core
```

Spring MVC makes it easier to handle web requests.

Instead of writing Servlet code directly, we can use clean annotations and controller classes.

For example, later we will write code like:

```java
@GetMapping("/hello")
public String sayHello() {
    return "Hello World";
}
```

Spring MVC internally uses Servlet technology, but it gives developers a cleaner programming model.

---

20. Spring Data

Most applications need to store data permanently.

For that, applications usually need a database.

Earlier, Java developers commonly used **JDBC**.

With JDBC, developers had to:

```
Write SQL manually
Open database connections
Execute queries
Handle result sets
Close resources
Manage repetitive database code
```

Later, frameworks like **Hibernate** made database work easier.

Hibernate maps Java objects to database tables.

This concept is called **Object-Relational Mapping**, or ORM.

JPA and Hibernate

JPA stands for:

```
Java Persistence API
```

JPA is a specification.

That means it defines rules and guidelines for how Java objects should be mapped to database tables.

But JPA itself does not provide the actual working implementation.

Hibernate is one of the most popular implementations of JPA.

In simple words:

```
JPA tells what should be done.
Hibernate actually does it.
```

Spring Data JPA goes one step further and reduces even more boilerplate code.

The flow looks like this:

```
Spring Data JPA → Hibernate → JDBC → Database
```

A simple way to remember:

> JDBC crawled so Hibernate could walk, and Hibernate walked so Spring Data JPA could fly.

---

21. Spring Security

Spring Security is used for authentication and authorization.

It helps with features like:

```
Login
JWT
OAuth
Roles
Permissions
Password encoding
CSRF protection
Access control
```

Without Spring Security, developers would have to manually write a lot of sensitive and repetitive security code.

Spring Security gives a standard and powerful way to secure Java applications.

---

22. Spring AOP

AOP stands for:

```
Aspect-Oriented Programming
```

Spring AOP helps us separate cross-cutting concerns from business logic.

Examples of cross-cutting concerns:

```
Logging
Security checks
Transaction management
Performance tracking
Exception handling
```

These are things that may be needed across many parts of an application.

We will study AOP properly later in the series.

---

23. Spring AI

Spring AI is a newer part of the Spring ecosystem.

It helps Java developers integrate AI features into Spring applications.

It can work with:

```
OpenAI
Gemini
Anthropic
Vector databases
RAG systems
Embeddings
AI chat models
```

This is useful when building AI-powered applications using Java and Spring.

---

24. What is Spring Boot?

Spring Boot is not a replacement for Spring.

Spring Boot is an automation layer on top of Spring.

It helps developers create Spring applications faster by providing:

```
Auto-configuration
Starter dependencies
Embedded servers
Sensible defaults
Production-ready features
Less manual configuration
```

In simple words:

> Spring Boot configures Spring for us so we can start building applications quickly.

---

25. Spring Boot is Opinionated

Spring Boot makes many default assumptions.

These assumptions are called **opinions**.

An opinionated framework provides sensible defaults so developers do not have to configure everything manually.

For example, if we add a web dependency, Spring Boot assumes that we want to build a web application.

So it can automatically configure things like:

```
Embedded Tomcat
Spring MVC setup
Default application structure
JSON support
Basic error handling
```

This saves a lot of time.

But the real skills are still in understanding:

```
Spring Core
Spring MVC
Dependency Injection
IoC
Beans
```

```
Servlets
HTTP
Database concepts
Security concepts
```

Spring Boot makes development faster, but Spring fundamentals make you a stronger developer.

---

26. Spring Boot vs Spring Framework

A common confusion is:

> Are Spring and Spring Boot the same?

No.

Spring Framework provides the core features and different modules.

Spring Boot makes Spring easier to use by reducing configuration and setup work.

Simple comparison:

Spring Framework	Spring Boot
Provides core features and modules	Provides auto-configuration and quick setup
Requires more manual configuration	Reduces manual configuration
Gives flexibility	Gives sensible defaults
Foundation of the ecosystem	Built on top of Spring

So we can say:

```
Spring Boot uses Spring.
Spring Boot does not replace Spring.
```

---

27. Where Do Microservices Fit?

Microservices are not a separate Spring module.

Microservices are an **architecture style**.

In a microservices architecture, a large application is divided into smaller independent services.

For example:

```
User Service
Order Service
Payment Service
Notification Service
Product Service
```

Each service can be developed, deployed, and scaled independently.

Spring Boot is commonly used to build microservices because it makes it easy to create independent production-ready applications.

So the idea is:

```
Microservices = Architecture style
Spring Boot = Tool commonly used to build them
```

---

28. Complete Flow: From Browser to Spring Boot

Now we can connect the whole journey.

```
Browser sends HTTP request
        |
        v
Servlet Container receives request
        |
        v
Servlet technology handles web communication
        |
        v
Spring MVC gives a cleaner web programming model
        |
        v
Spring Core manages objects and dependencies
        |
        v
```

```
Spring Boot auto-configures everything
        |
        v
Developer writes business logic
```

This is the bigger picture behind modern Java backend development.

---

29. Why Understanding This History Matters

Many beginners start directly with Spring Boot and write code like:

```
@RestController
@GetMapping("/hello")
```

The code works, but they do not understand what is happening behind the scenes.

When we understand the journey from:

```
Client–Server Architecture
HTTP
JVM
Core Java networking
Servlets
Spring Framework
Spring Boot
```

Spring Boot no longer feels magical.

It starts making sense.

We understand why each technology came into the picture and what problem it solved.

---

30. Final Summary

The complete evolution looks like this:

```
Client–Server Architecture
        ↓
HTTP communication
        ↓
Core Java limitation for web apps
        ↓
Java networking with sockets
        ↓
Manual HTTP parsing problem
        ↓
Servlets and Servlet Containers
        ↓
Spring Framework
        ↓
Spring ecosystem
        ↓
Spring Boot
```

Key takeaways:

```
Client sends a request.
Server sends a response.
HTTP defines the communication format.
Core Java does not automatically understand HTTP.
Servlets helped Java handle web requests.
Servlet Containers handle low-level web work.
Spring made enterprise Java development cleaner.
Spring Core manages objects and dependencies.
Spring MVC simplifies web development.
Spring Data simplifies database access.
Spring Security handles authentication and authorization.
Spring Boot auto-configures Spring applications.
Microservices are an architecture style, not a Spring module.
```

The goal of this series is not just to write Spring Boot code.

The goal is to understand how modern Java backend development actually works.

Once this foundation is clear, Spring Boot becomes much easier to learn, debug, and use in real projects.